Gohil Amarsinh M.

DATABASE SYSTEMS BOOTCAMP

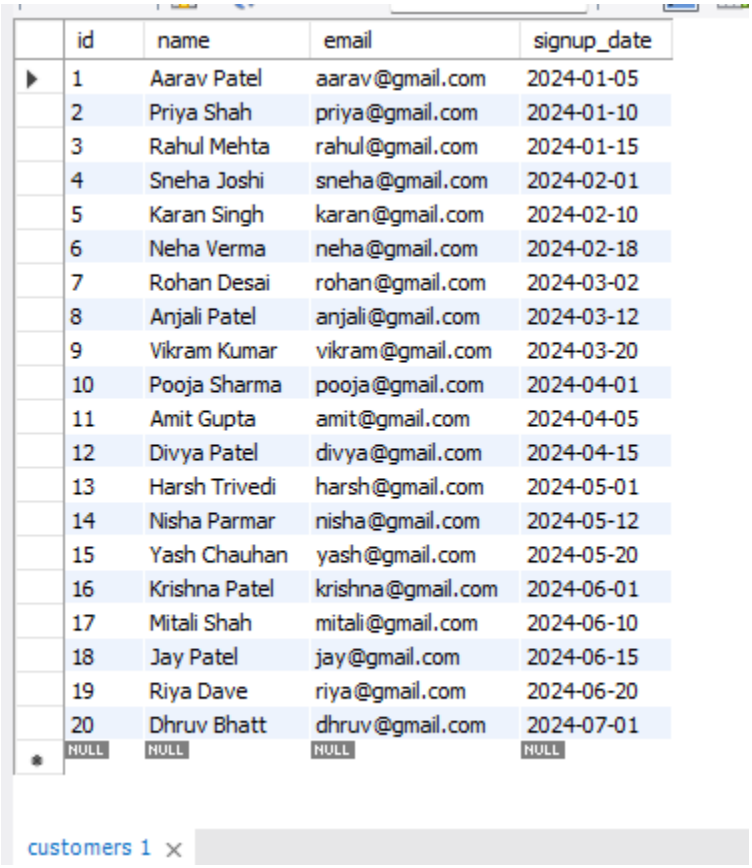
Practice Assignment

Transactions · Joins · Indexing · Query Optimization

Part 1 — Build Practice Schema

Table	Columns	Notes
customers	id (primary key), name, email (unique), signup_date	Add at least 15–20 sample customers.
restaurants	id (primary key), name, cuisine	Add at least 8–10 sample restaurants.
orders	id (primary key), customer_id, restaurant_id, total, status, created_at	Add at least 40–50 sample orders. customer_id and restaurant_id must match real ids from your other two tables.
accounts	id (primary key), name, balance	Just 2 rows — e.g. one account named Alice and one named Bob — used only for Task 1.

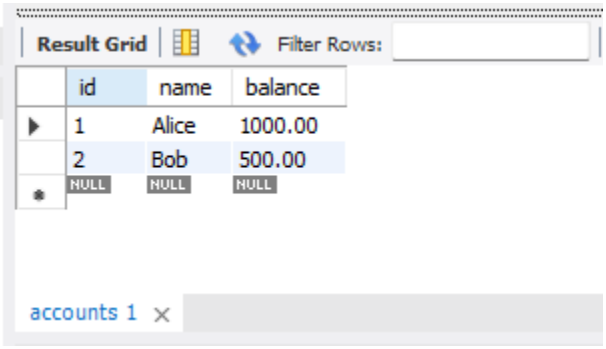
Customers table :



id	name	email	signup_date
1	Aarav Patel	aarav@gmail.com	2024-01-05
2	Priya Shah	priya@gmail.com	2024-01-10
3	Rahul Mehta	rahul@gmail.com	2024-01-15
4	Sneha Joshi	sneha@gmail.com	2024-02-01
5	Karan Singh	karan@gmail.com	2024-02-10
6	Neha Verma	neha@gmail.com	2024-02-18
7	Rohan Desai	rohan@gmail.com	2024-03-02
8	Anjali Patel	anjali@gmail.com	2024-03-12
9	Vikram Kumar	vikram@gmail.com	2024-03-20
10	Pooja Sharma	pooja@gmail.com	2024-04-01
11	Amit Gupta	amit@gmail.com	2024-04-05
12	Divya Patel	divya@gmail.com	2024-04-15
13	Harsh Trivedi	harsh@gmail.com	2024-05-01
14	Nisha Parmar	nisha@gmail.com	2024-05-12
15	Yash Chauhan	yash@gmail.com	2024-05-20
16	Krishna Patel	krishna@gmail.com	2024-06-01
17	Mitali Shah	mitali@gmail.com	2024-06-10
18	Jay Patel	jay@gmail.com	2024-06-15
19	Riya Dave	riya@gmail.com	2024-06-20
20	Dhruv Bhatt	dhruv@gmail.com	2024-07-01
NULL	NULL	NULL	NULL

customers 1 ×

Accounts table:

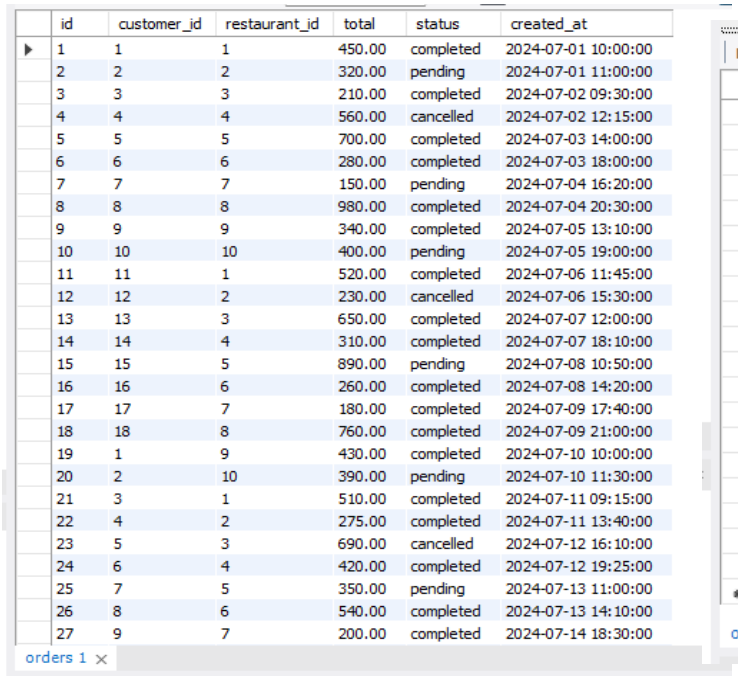


Result Grid | Filter Rows:

id	name	balance
1	Alice	1000.00
2	Bob	500.00
NULL	NULL	NULL

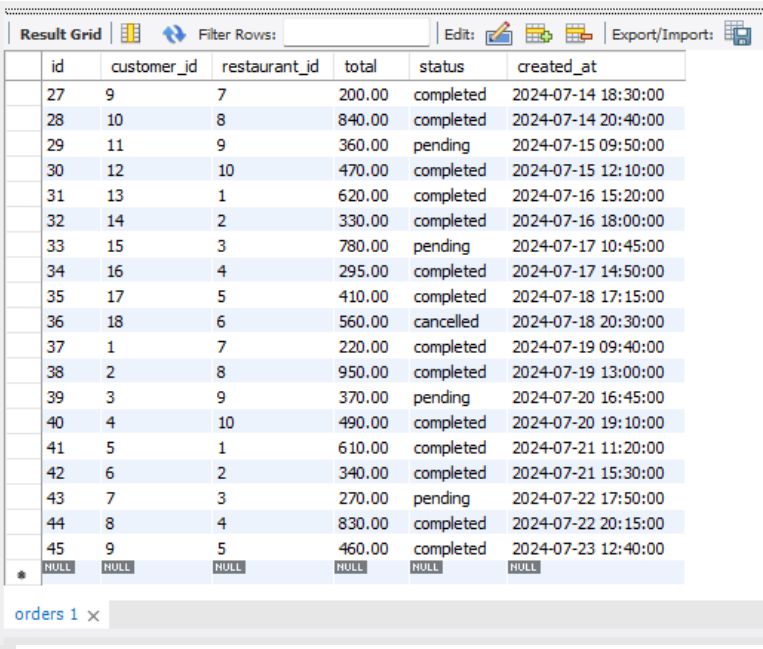
accounts 1

Orders table:



id	customer_id	restaurant_id	total	status	created_at
1	1	1	450.00	completed	2024-07-01 10:00:00
2	2	2	320.00	pending	2024-07-01 11:00:00
3	3	3	210.00	completed	2024-07-02 09:30:00
4	4	4	560.00	cancelled	2024-07-02 12:15:00
5	5	5	700.00	completed	2024-07-03 14:00:00
6	6	6	280.00	completed	2024-07-03 18:00:00
7	7	7	150.00	pending	2024-07-04 16:20:00
8	8	8	980.00	completed	2024-07-04 20:30:00
9	9	9	340.00	completed	2024-07-05 13:10:00
10	10	10	400.00	pending	2024-07-05 19:00:00
11	11	1	520.00	completed	2024-07-06 11:45:00
12	12	2	230.00	cancelled	2024-07-06 15:30:00
13	13	3	650.00	completed	2024-07-07 12:00:00
14	14	4	310.00	completed	2024-07-07 18:10:00
15	15	5	890.00	pending	2024-07-08 10:50:00
16	16	6	260.00	completed	2024-07-08 14:20:00
17	17	7	180.00	completed	2024-07-09 17:40:00
18	18	8	760.00	completed	2024-07-09 21:00:00
19	1	9	430.00	completed	2024-07-10 10:00:00
20	2	10	390.00	pending	2024-07-10 11:30:00
21	3	1	510.00	completed	2024-07-11 09:15:00
22	4	2	275.00	completed	2024-07-11 13:40:00
23	5	3	690.00	cancelled	2024-07-12 16:10:00
24	6	4	420.00	completed	2024-07-12 19:25:00
25	7	5	350.00	pending	2024-07-13 11:00:00
26	8	6	540.00	completed	2024-07-13 14:10:00
27	9	7	200.00	completed	2024-07-14 18:30:00

orders 1

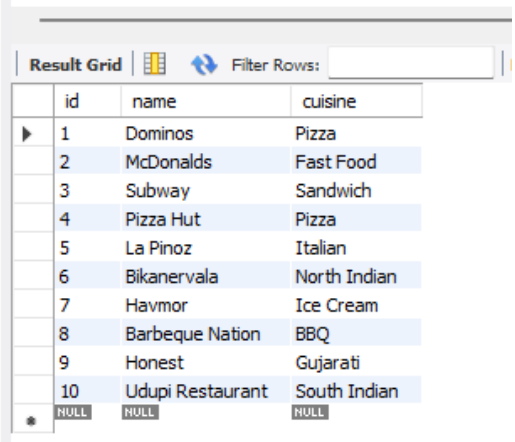


Result Grid | Filter Rows: | Edit: | Export/Import:

id	customer_id	restaurant_id	total	status	created_at
27	9	7	200.00	completed	2024-07-14 18:30:00
28	10	8	840.00	completed	2024-07-14 20:40:00
29	11	9	360.00	pending	2024-07-15 09:50:00
30	12	10	470.00	completed	2024-07-15 12:10:00
31	13	1	620.00	completed	2024-07-16 15:20:00
32	14	2	330.00	completed	2024-07-16 18:00:00
33	15	3	780.00	pending	2024-07-17 10:45:00
34	16	4	295.00	completed	2024-07-17 14:50:00
35	17	5	410.00	completed	2024-07-18 17:15:00
36	18	6	560.00	cancelled	2024-07-18 20:30:00
37	1	7	220.00	completed	2024-07-19 09:40:00
38	2	8	950.00	completed	2024-07-19 13:00:00
39	3	9	370.00	pending	2024-07-20 16:45:00
40	4	10	490.00	completed	2024-07-20 19:10:00
41	5	1	610.00	completed	2024-07-21 11:20:00
42	6	2	340.00	completed	2024-07-21 15:30:00
43	7	3	270.00	pending	2024-07-22 17:50:00
44	8	4	830.00	completed	2024-07-22 20:15:00
45	9	5	460.00	completed	2024-07-23 12:40:00
NULL	NULL	NULL	NULL	NULL	NULL

orders 1

Restaurants table:



Result Grid | Filter Rows:

id	name	cuisine
1	Dominos	Pizza
2	McDonalds	Fast Food
3	Subway	Sandwich
4	Pizza Hut	Pizza
5	La Pinoz	Italian
6	Bikanervala	North Indian
7	Havmor	Ice Cream
8	Barbeque Nation	BBQ
9	Honest	Gujarati
10	Udupi Restaurant	South Indian
NULL	NULL	NULL

Part 2 — Tasks

TASK 1 **Transactions (ACID)**

Using your accounts table:

a) Write a transaction that transfers ₹200 from one account to the other, but ends in ROLLBACK. Confirm both balances are unchanged afterward.

b) Write the same transfer again, this time ending in COMMIT. Confirm both balances have changed.

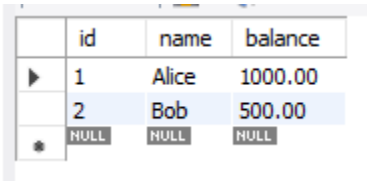
c) In 2–3 sentences, explain which ACID property guarantees the money is never "lost" in the middle of a transfer.

Answer :

The Atomicity property of ACID guarantees that money is never lost during a transaction. It follows the "all-or-nothing" rule, meaning both the debit and credit operations must complete successfully together. If a failure occurs in the middle of the transaction, the database automatically rolls back all changes, keeping the data consistent.

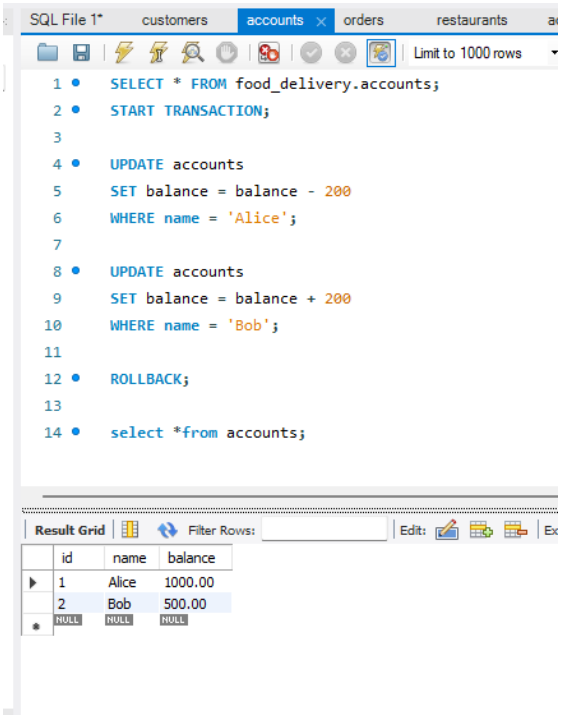
Output:

(before transaction)



id	name	balance
1	Alice	1000.00
2	Bob	500.00
NULL	NULL	NULL

(after ROLLBACK)



(after COMMIT)

```
4
5  START TRANSACTION;
6
7  UPDATE accounts
8  SET balance = balance - 200
9  WHERE name = 'Alice';
10
11 UPDATE accounts
12 SET balance = balance + 200
13 WHERE name = 'Bob';
14
15 COMMIT;
16
17 SELECT * FROM accounts;
18
```

Result Grid | Filter Rows: | Edit:

id	name	balance
1	Alice	800.00
2	Bob	700.00
NULL	NULL	NULL

TASK 2 **Joins**

Using your customers and orders tables:

a) Write an INNER JOIN that lists every customer who has placed at least one order.

b) Write a LEFT JOIN that lists every customer, including those with zero orders. Then add a WHERE clause to isolate only the customers who have never ordered.

c) In one sentence, explain why a marketing team would prefer the result of query (b) over query (a).

(a): INNER JOIN :

```
4
5  SELECT
6      c.id,
7      c.name,
8      c.email,
9      o.id AS order_id,
10     o.total,
11     o.status
12 FROM customers c
13 INNER JOIN orders o
14 ON c.id = o.customer_id;
15
```

id	name	email	order_id	total	status
1	Aarav Patel	aarav@gmail.com	1	450.00	completed
1	Aarav Patel	aarav@gmail.com	19	430.00	completed
1	Aarav Patel	aarav@gmail.com	37	220.00	completed
2	Priya Shah	priya@gmail.com	2	320.00	pending
2	Priya Shah	priya@gmail.com	20	390.00	pending
2	Priya Shah	priya@gmail.com	38	950.00	completed
3	Rahul Mehta	rahul@gmail.com	3	210.00	completed
3	Rahul Mehta	rahul@gmail.com	21	510.00	completed
3	Rahul Mehta	rahul@gmail.com	39	370.00	pending
4	Sneha Joshi	sneha@gmail.com	4	560.00	cancelled
4	Sneha Joshi	sneha@gmail.com	22	275.00	completed
4	Sneha Joshi	sneha@gmail.com	40	490.00	completed
5	Karan Singh	karan@gmail.com	5	700.00	completed
5	Karan Singh	karan@gmail.com	23	690.00	cancelled
5	Karan Singh	karan@gmail.com	41	610.00	completed
6	Neha Verma	neha@gmail.com	6	280.00	completed
6	Neha Verma	neha@gmail.com	24	420.00	completed
6	Neha Verma	neha@gmail.com	42	340.00	completed
7	Rohan Desai	rohan@gmail.com	7	150.00	pending
7	Rohan Desai	rohan@gmail.com	25	350.00	pending
7	Rohan Desai	rohan@gmail.com	43	270.00	pending

Result 27 ×

(b): LEFT JOIN :

```
5  SELECT
6      c.id,
7      c.name,
8      c.email,
9      o.id AS order_id
10 FROM customers c
11 LEFT JOIN orders o
12 ON c.id = o.customer_id;
13
```

Result Grid | Filter Rows: | Expor

id	name	email	order_id
1	Aarav Patel	aarav@gmail.com	1
1	Aarav Patel	aarav@gmail.com	19
1	Aarav Patel	aarav@gmail.com	37
2	Priya Shah	priya@gmail.com	2
2	Priya Shah	priya@gmail.com	20
2	Priya Shah	priya@gmail.com	38
3	Rahul Mehta	rahul@gmail.com	3
3	Rahul Mehta	rahul@gmail.com	21
3	Rahul Mehta	rahul@gmail.com	39
4	Sneha Joshi	sneha@gmail.com	4
4	Sneha Joshi	sneha@gmail.com	22
4	Sneha Joshi	sneha@gmail.com	40
5	Karan Singh	karan@gmail.com	5
5	Karan Singh	karan@gmail.com	23
5	Karan Singh	karan@gmail.com	41
6	Neha Verma	neha@gmail.com	6
6	Neha Verma	neha@gmail.com	24
6	Neha Verma	neha@gmail.com	42
7	Rohan Desai	rohan@gmail.com	7
7	Rohan Desai	rohan@gmail.com	25
7	Rohan Desai	rohan@gmail.com	43

(c): Written Answer :

The marketing team prefers the result of the LEFT JOIN query because it identifies customers who have never placed an order. These customers can be targeted with discounts, coupons, or promotional offers to encourage them to make their first purchase.

TASK 3 **Indexing**

Using your orders table:

a) Pick a column that is not currently indexed (for example status or created_at).

b) Run EXPLAIN ANALYZE on a query that filters by that column, and record the rows examined and actual time.

c) Create an index on that column.

d) Re-run the exact same query and record the new numbers.

e) Submit a one-line comparison of before vs. after.

(a): Choose a Column

We'll use the **status** column in the orders table because it is commonly used for filtering. This also matches your reference solution

(b): Before Creating the Index

```
5 ●    EXPLAIN ANALYZE
6      SELECT *
7      FROM orders
8      WHERE status = 'completed';
```

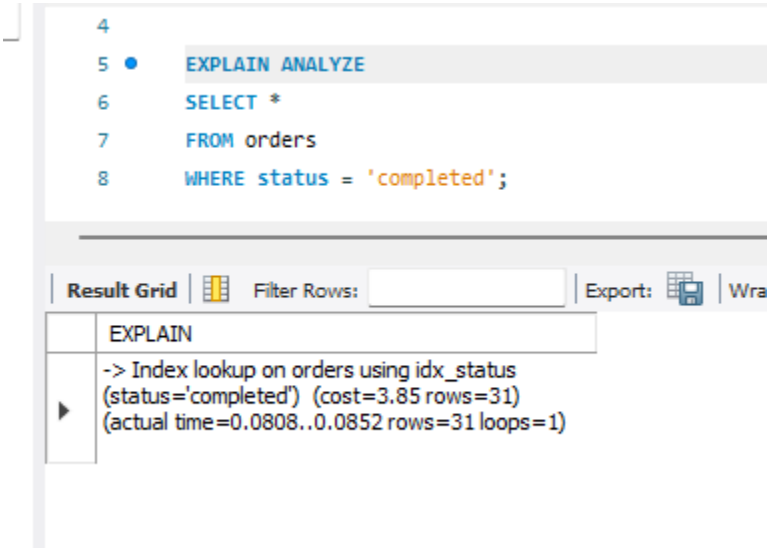
Result Grid | Filter Rows: | Export: | Wra

EXPLAIN
-> Filter: (orders.`status` = 'completed') (cost=4.75 rows=4.5) (actual time=0.0312..0.0436 rows=31 loops=1)
-> Table scan on orders (cost=4.75 rows=45) (actual time=0.0291..0.0383 rows=45 loops=1)

(c): Create the Index

```
CREATE INDEX idx_status
ON orders(status);
```

(d): Run the Same Query Again



(e): Written Answer

Before creating the index, MySQL scanned the entire orders table to find rows with status = 'completed'. After creating the index, MySQL can find matching rows using the index, making the query more efficient, especially as the number of records increases.

TASK 4 **Query Optimization**

Start with this query, rewritten to use your own table and column names:

```
SELECT * FROM orders WHERE customer_id IN (
  SELECT id FROM customers WHERE signup_date >= 'some date'
);
```

a) Rewrite it as a JOIN instead of a subquery.

b) Rewrite it again to select only the specific columns you'd actually need on a screen (avoid SELECT *).

c) Run EXPLAIN ANALYZE on your original query and your final rewritten version, and note what changed.

(a): Rewrite Using a JOIN

```sql
SELECT *
FROM orders o
JOIN customers c
ON o.customer_id = c.id
WHERE c.signup_date >= '2024-04-01';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	customer_id	restaurant_id	total	status	created_at	id	name	email	signup_date
10	10	10	400.00	pending	2024-07-05 19:00:00	10	Pooja Sharma	pooja@gmail.com	2024-04-01
28	10	8	840.00	completed	2024-07-14 20:40:00	10	Pooja Sharma	pooja@gmail.com	2024-04-01
11	11	1	520.00	completed	2024-07-06 11:45:00	11	Amit Gupta	amit@gmail.com	2024-04-05
29	11	9	360.00	pending	2024-07-15 09:50:00	11	Amit Gupta	amit@gmail.com	2024-04-05
12	12	2	230.00	cancelled	2024-07-06 15:30:00	12	Divya Patel	divya@gmail.com	2024-04-15
30	12	10	470.00	completed	2024-07-15 12:10:00	12	Divya Patel	divya@gmail.com	2024-04-15
13	13	3	650.00	completed	2024-07-07 12:00:00	13	Harsh Trivedi	harsh@gmail.com	2024-05-01
31	13	1	620.00	completed	2024-07-16 15:20:00	13	Harsh Trivedi	harsh@gmail.com	2024-05-01
14	14	4	310.00	completed	2024-07-07 18:10:00	14	Nisha Parmar	nisha@gmail.com	2024-05-12
32	14	2	330.00	completed	2024-07-16 18:00:00	14	Nisha Parmar	nisha@gmail.com	2024-05-12
15	15	5	890.00	pending	2024-07-08 10:50:00	15	Yash Chauhan	yash@gmail.com	2024-05-20
33	15	3	780.00	pending	2024-07-17 10:45:00	15	Yash Chauhan	yash@gmail.com	2024-05-20
16	16	6	260.00	completed	2024-07-08 14:20:00	16	Krishna Patel	krishna@gmail.com	2024-06-01
34	16	4	295.00	completed	2024-07-17 14:50:00	16	Krishna Patel	krishna@gmail.com	2024-06-01
17	17	7	180.00	completed	2024-07-09 17:40:00	17	Mitali Shah	mitali@gmail.com	2024-06-10
35	17	5	410.00	completed	2024-07-18 17:15:00	17	Mitali Shah	mitali@gmail.com	2024-06-10
18	18	8	760.00	completed	2024-07-09 21:00:00	18	Jay Patel	jay@gmail.com	2024-06-15
36	18	6	560.00	cancelled	2024-07-18 20:30:00	18	Jay Patel	jay@gmail.com	2024-06-15

(b): Select Only Required Columns

```sql
SELECT
    o.id AS order_id,
    c.name,
    o.total,
    o.status,
    o.created_at
FROM orders o
JOIN customers c
ON o.customer_id = c.id
WHERE c.signup_date >= '2024-04-01';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conte

order_id	name	total	status	created_at
10	Pooja Sharma	400.00	pending	2024-07-05 19:00:00
28	Pooja Sharma	840.00	completed	2024-07-14 20:40:00
11	Amit Gupta	520.00	completed	2024-07-06 11:45:00
29	Amit Gupta	360.00	pending	2024-07-15 09:50:00
12	Divya Patel	230.00	cancelled	2024-07-06 15:30:00
30	Divya Patel	470.00	completed	2024-07-15 12:10:00
13	Harsh Trivedi	650.00	completed	2024-07-07 12:00:00
31	Harsh Trivedi	620.00	completed	2024-07-16 15:20:00
14	Nisha Parmar	310.00	completed	2024-07-07 18:10:00
32	Nisha Parmar	330.00	completed	2024-07-16 18:00:00
15	Yash Chauhan	890.00	pending	2024-07-08 10:50:00
33	Yash Chauhan	780.00	pending	2024-07-17 10:45:00
16	Krishna Patel	260.00	completed	2024-07-08 14:20:00
34	Krishna Patel	295.00	completed	2024-07-17 14:50:00
17	Mitali Shah	180.00	completed	2024-07-09 17:40:00
35	Mitali Shah	410.00	completed	2024-07-18 17:15:00
18	Jay Patel	760.00	completed	2024-07-09 21:00:00
36	Jay Patel	560.00	cancelled	2024-07-18 20:30:00

(c): Compare Execution Plans

Original Query

```
5  EXPLAIN ANALYZE
6  SELECT *
7  FROM orders
8  WHERE customer_id IN (
9      SELECT id
10     FROM customers
11     WHERE signup_date >= '2024-04-01'
12 );
```

Result Grid | Filter Rows: | Export:

EXPLAIN
-> Nested loop inner join (cost=8.08 rows=16.7) (actual time=0.074..0.108 rows=18 loops=1)
-> Filter: (customers.signup_date >= DATE'2024-04-01') (cost=2.25 rows=6.67) (actual time=0.0418..0.0468 rows=11 loops=1)
-> Table scan on customers ...

Optimized Query

```
5  EXPLAIN ANALYZE
6  SELECT
7      o.id AS order_id,
8      c.name,
9      o.total,
10     o.status,
11     o.created_at
12 FROM orders o
13 JOIN customers c
14 ON o.customer_id = c.id
15 WHERE c.signup_date >= '2024-04-01';
```

Result Grid | Filter Rows: | Export

EXPLAIN
-> Nested loop inner join (cost=8.08 rows=16.7) (actual time=0.049..0.0793 rows=18 loops=1)
-> Filter: (c.signup_date >= DATE'2024-04-01') (cost=2.25 rows=6.67) (actual time=0.0321..0.0357 rows=11 loops=1)
-> Table scan on c (cost=2.25 row...

The optimized query uses a JOIN instead of a subquery and selects only the required columns instead of using SELECT *. This reduces unnecessary data retrieval and can improve query performance, especially when working with large tables.

Final Challenge — A Real-World Problem

Concepts used: *this task combines everything from Tasks 2, 3, and 4 — Joins, Indexing, and Query Optimization — in one realistic scenario.*

TASK 5 The Customer Profile Page

Scenario: a food-delivery company shows each customer their 5 most recent orders on their profile page — the order total, status, and the name of the restaurant it came from. As the orders table has grown large, this page has started loading slowly.

Complete the following steps, in order:

- a) Write a query that joins orders with restaurants (and customers, if needed) to return: restaurant name, order total, order status, and created_at — filtered to one specific customer, sorted by the most recent order first, limited to 5 results.
- b) Run EXPLAIN ANALYZE on it and record the rows examined and time taken.
- c) Create the index (or indexes) you think will make this query fast. Think about which column(s) appear in your WHERE clause and your ORDER BY.
- d) Re-run the same query and compare the new numbers to step (b).
- e) Rewrite the query to select only the exact columns the profile page needs — no SELECT *.
- f) Submit your before/after comparison, plus 2–3 sentences explaining why the index you chose helped.

Task 5(a)

Query to Display the Latest 5 Orders

```sql
SELECT
    r.name AS restaurant_name,
    o.total,
    o.status,
    o.created_at
FROM orders o
JOIN restaurants r
ON o.restaurant_id = r.id
WHERE o.customer_id = 1
ORDER BY o.created_at DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

restaurant_name	total	status	created_at
Havmor	220.00	completed	2024-07-19 09:40:00
Honest	430.00	completed	2024-07-10 10:00:00
Dominos	450.00	completed	2024-07-01 10:00:00

Task 5(b)

```sql
EXPLAIN ANALYZE
SELECT
    r.name AS restaurant_name,
    o.total,
    o.status,
    o.created_at
FROM orders o
JOIN restaurants r
ON o.restaurant_id = r.id
WHERE o.customer_id = 1
ORDER BY o.created_at DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export:

EXPLAIN
-> Limit: 5 row(s) (cost=2.1 rows=3) (actual time=0.0659..0.0682 rows=3 loops=1) -> Nested loop inner join (cost=2.1 rows=3) (actual time=0.0649..0.0671 rows=3 loops=1) -> Sort: o.created_at DESC (cost=1.05 rows=3) (actual time=0.0572..0.0...

Task 5(c)

Create an index.

```
CREATE INDEX idx_customer_created
ON orders(customer_id, created_at);
```

Task 5(d)

Run the same query again.

```
EXPLAIN ANALYZE
SELECT
    r.name AS restaurant_name,
    o.total,
    o.status,
    o.created_at
FROM orders o
JOIN restaurants r
ON o.restaurant_id = r.id
WHERE o.customer_id = 1
ORDER BY o.created_at DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export:

EXPLAIN
-> Limit: 5 row(s) (cost=2.1 rows=3) (actual time=0.0423..0.0462 rows=3 loops=1) -> Nested loop inner join (cost=2.1 rows=3) (actual time=0.0412..0.045 rows=3 loops=1) -> Index lookup on o using idx_customer_created (customer_id=1) (reverse...

Task 5(e)

```
SELECT
    r.name AS restaurant_name,
    o.total,
    o.status,
    o.created_at
FROM orders o
INNER JOIN restaurants r
    ON o.restaurant_id = r.id
WHERE o.customer_id = 1
ORDER BY o.created_at DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap

restaurant_name	total	status	created_at
Havmor	220.00	completed	2024-07-19 09:40:00
Honest	430.00	completed	2024-07-10 10:00:00
Dominos	450.00	completed	2024-07-01 10:00:00

Task 5(f)

Before vs After Comparison

Before creating the index, MySQL scanned more rows to find the customer's orders and then sorted them by date. After creating the composite index on (customer_id, created_at), MySQL could locate the customer's records more efficiently and retrieve the latest five orders faster.

Explanation

The composite index helps because the query filters by customer_id and sorts by created_at. Since both columns are included in the same index, MySQL can quickly locate the customer's orders in the correct order, reducing the amount of data it needs to scan and sort.